

Computer Security Exam

Professors M. Carminati & S. Zanero

Milan, 09/09/2019

Last (family) Name _____

First (given) Name _____

Matricola or Codice Persona _____

Have you done any challenges/homework, even partially?	☐ Yes	☐ No
Professor:	☐ Carminati	☐ Zanero
Course:	☐ Milan	☐ Como

Instructions

- **Duration**: 2 hours
- The exam is composed of 16 pages. Check that you have all of them.
- Just as a cross check, tell us whether you have completed the homework, by putting an "X" mark appropriately.
- The exam is "closed books". Please put away in a non-suspicious place (i.e. not below the desk) any notebook, or similar. You will be expelled if, at any time, if you do not follow this rule.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Shut down and store electronic devices. They will be subject to inspection if found and you may be expelled if you are found using one.
- Please answer within the allowed space. Schemes are good, short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- The answers should be written exclusively in the space provided below the questions.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam.**

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

SOLUTION

Answer provided in this solution **MUST BE CONSIDERED ONLY AS A HINT**
for the correct answer, and they are not necessarily complete.

Exercise 1 (12 points)

Consider the C program below (vuln_prog.c).

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04
05  void guess(int n1, char *s1) {
06      struct {
07          char buf[8];
08          char flag[12];
09          char canary[4]="LOL";
10      } s;
11
12      if (strlen(s1) >= n1 && n1 < 4) {
13          printf("Insert your secret:");
14          snprintf(s.buf, 40, s1);
15          buf[3] = canary[n1];
16          buf[7] = "\0";
17
18          if (strncmp(s.canary, "LOL", 3) != 0)
19              abort();
20          else
21              printf("Welcome: %s", s.buf);
22      } else {
23          printf(s1);
24      }
25  }
26
27  int main(int argc, char** argv) {
28
29      guess(atoi(argv[1]), argv[2]);
30  }
```

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the (exact) address of the function `system()` is `0xfce48da0`.
- Environment variables are located in the highest addresses.

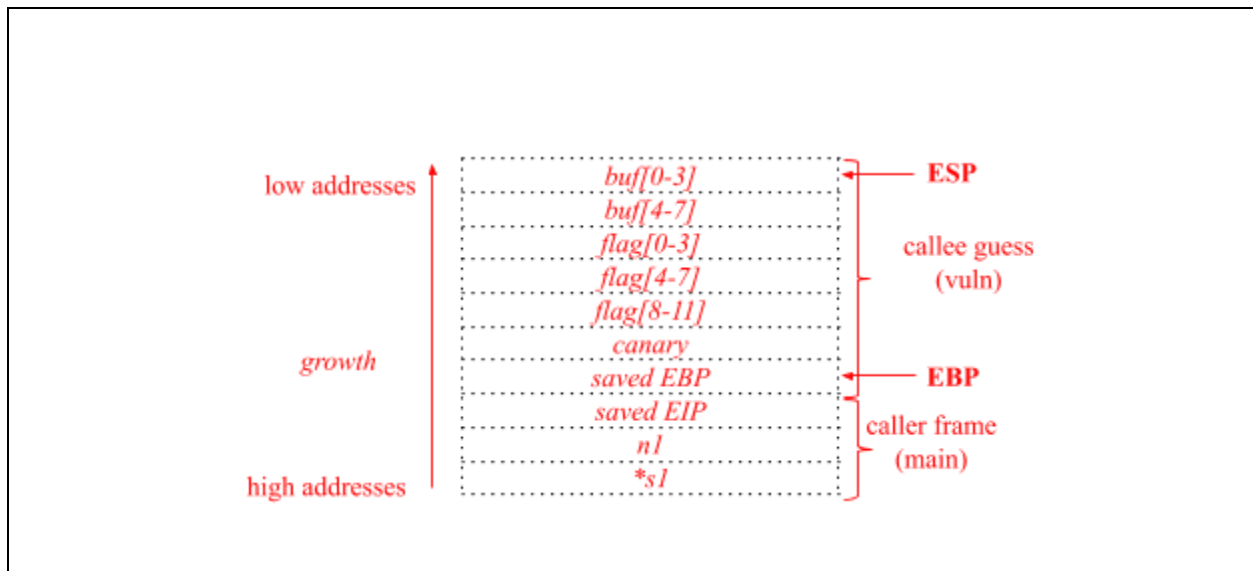
1. [1 point] The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table.

Vulnerability	Line	Motivation
Buffer Overflow	14	See slides
Format String	14, 23	See slides

2. [1 point] Assuming that the program is compiled and run for the usual IA-32 architecture (32-bits), with the usual cdecl calling convention, draw the stack layout just before the execution of line 11, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable (i.e., EBP, ESP);
- The content of relevant registers;
- The boundaries of the function stack frames (main and guess)

Show also the content of the caller frame (you can ignore the environment variables: just focus on what matters for the exploitation of typical memory corruption vulnerabilities).



4. Focus only on the Buffer Overflow vulnerability you identified.

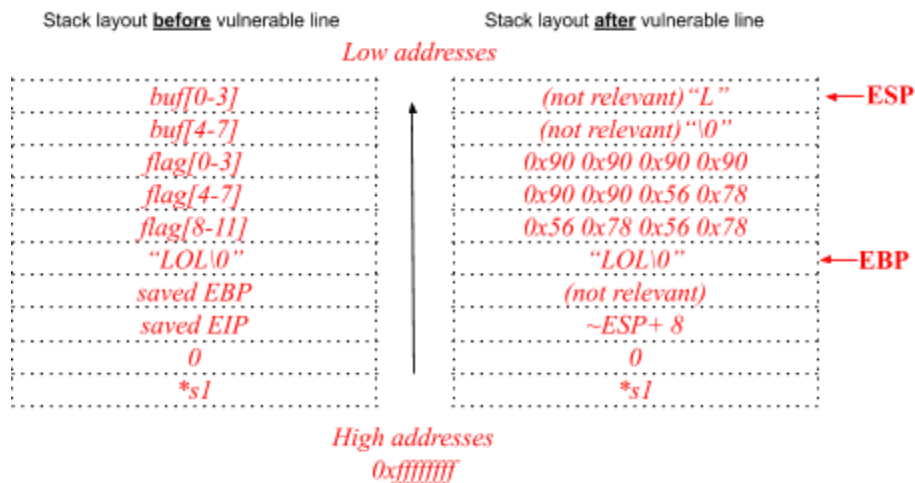
4.1 [2 point] Assume that the program is compiled and run with no mitigation against exploitation of memory corruption vulnerabilities (**no canary, executable stack**, environment **with no ASLR** active).

Write an exploit for the buffer overflow vulnerability identified to bypass the authentication mechanism. Your exploit **must** execute the following simple shellcode composed only by 6 instructions (6 bytes) that launch the program /bin/secret: 0x56 0x78 0x56 0x78 0x56 0x78.

Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation.

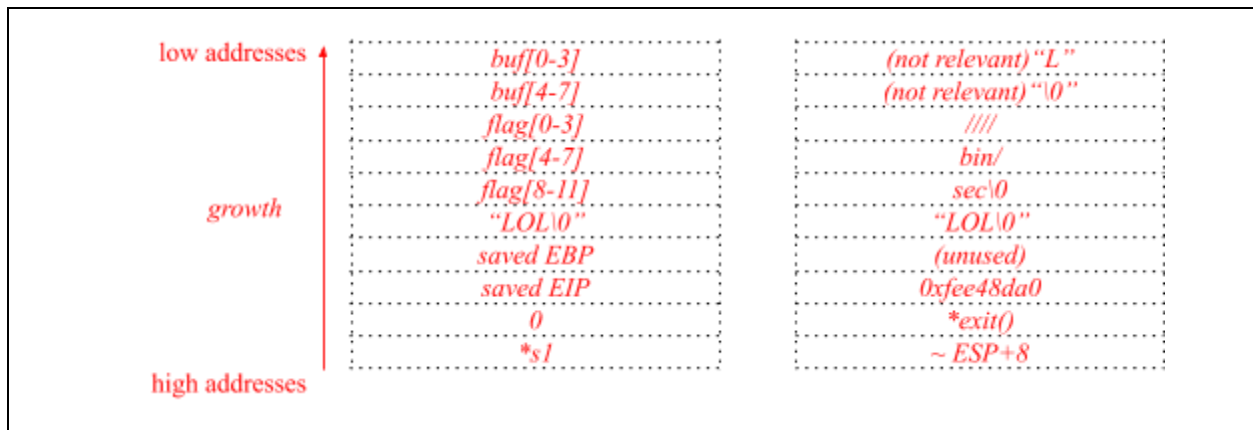
Command line argument (argv[1]): 0

Command line argument (argv[2]): **your exploit**



4.2 [1 point] Now assume that the stack is not executable. Is the exploit you just wrote still working without modifications? If not, explain why and propose an alternative solution to launch the program /bin/secret. Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation.

We write in the flag the string /bin/sec, and we overwrite the saved EIP with the address of system(). We then overwrite 's1' with a pointer to ~ESP+8 so that, when jumping into the system() function, this pointer will be at ESP-4 (i.e., where system() expects the parameter).



5. [1 point] This time focus only on the format string vulnerability you identified. Assume that ASLR is not active and that you were able to find this working exploit to execute a piece of shellcode saved in an environment variable.

```
./vuln_prog 10 `python -c 'print "\xba\x3a\x06\x43\xb8\x3a\x06\x43%31027c%6$hn%7595c%7$hn"'`
```

Clearly detail all the components of the format string exploit:

- The decimal components of the address of the environment variable containing your shellcode (i.e., **what to write**)
- The address of the saved return address (i.e., the address **where we want to write**)
- The **displacement on the stack** of your format string.

The command line argument is directly fed as a format string to sprintf, and the format string itself is on the stack with a given displacement: we just need to construct a standard format string exploit.

- We need to write 0x793b96e6 to 0x43063ab8 (<tgt>)
- 0x793b < 0x96e6 -> we write 0x793b first

The value of the command line argument will have the form

<tgt+2><tgt>%<N1>c%<pos>\$hn%<N2>c%<pos+1>\$hn

where

<tgt+2> = 0x43063aba

<tgt> = 0x43063ab8

<pos> = 6

<N1> = dec(0x793b) - 8 (bytes already written) = 31035 - 8 = 31027

<N2> = dec(0x4543) - 16961 (bytes already written) = 38630 - 31035 = 7595

6. [2 points] Consider **compiler level mitigations only**. Which of them are effective to mitigate:

- a) The detected buffer overflow vulnerability
- b) The detected format string vulnerability

Motivate your answer and briefly explain how they work.

[See Slides] for definitions. E.g., Canaries

a) yes, ...

b) no, ...

7. [2 points] Consider **code level mitigations only**. Which of them are effective to mitigate:

- c) The detected buffer overflow vulnerability
- d) The detected format string vulnerability

Motivate your answer and briefly explain how they work.

[See Slides] for definitions. E.g., Using safe(r) libraries

a) yes, ...

b) yes, ...

8. [2 points] Consider **OS level mitigations only**. Which of them are effective to mitigate:

- e) The detected buffer overflow vulnerability
- f) The detected format string vulnerability

Motivate your answer and briefly explain how they work.

[See Slides] for definitions. Non-executable stack, Address Space Layout Randomization (ASLR)

a) yes, ...

b) yes, ...

Exercise 2 (8 points)

Consider a webmail application (think about Gmail or Outlook.com), served from the domain <https://secure-webmail.example.com>.

The application uses the following code to manage the users' login:

```
00 var username = request.get['username'];
01 var password_hash = crypto.sha256(request.get['password']);
02
03 var q = db.query('SELECT * FROM users WHERE username = "'
04                 + username + '" AND password_hash = "' + password_hash + '" LIMIT 1');
05
06 if (q.get_num_rows() == 1) {
07     show_restricted_content(); // set user session and redirect to the home
08 } else {
09     error_http_403(); // unauthorized
10 }
```

Assume that `request.get` contains the variables passed through the GET HTTP request.

1. [1 point] Clearly, this code is affected by a SQL injection vulnerability. Briefly explain the vulnerable line/s of code and how it works in general.

SQL Injection. See slides for the explanation.

There must be a data flow from a **user-controlled HTTP variable** (e.g., parameter, cookie, or other header fields) to a **SQL query, without appropriate filtering and validation**. If this happens, the **SQL structure of the query can be modified**.

2. [1 point] Write an exploit for the SQL injection vulnerability that allows you to log in to the web application impersonating an existing and already registered user in the web application. Please write down all the steps and assumptions that you need for a successful exploitation.

Let's just make an HTTP request to /login with

username = 's.zanero' -- '

password = any value

*which will trigger the following query: SELECT * FROM users WHERE username = "s.zanero";--
(the rest of the query is commented out)*

This query returns one row if the user s.zanero is present in the DB, 0 otherwise, so this status can be inferred by the response (logged in = user in the DB, not logged in == user not in the DB).

After logging in, users can access their inbox and contact list, and read and compose emails. After login, users sessions are securely managed through session cookies (code not shown).

3 [1 point]. Consider the code that handles the display of an email message. Note that the body of the email message can contain HTML tags to specify basic message formatting (e.g., `<b>`, `<i>`, ...). The following PHP code snippet is used to display an email message:

```
<h1><?php echo htmlentities($subject); ?></h1>
<p><b>From: <?php echo htmlentities($from); ?></b></p>
<p><?php echo $body; ?></p>
```

Assume that `htmlentities()` replaces characters such as `<`, `>` with their corresponding HTML entities (`<`, `>`), and that `$subject`, `$from`, `$body` contain the subject, the sender IP address, and the body parsed from the message that is being displayed.

The code snippet clearly contains a XSS vulnerability. Considering the information you have, describe which subclass of XSS vulnerability it is (e.g., stored, reflected, DOM-based), how an attacker can exploit it, what goal can they achieve, and what is the simplest way to prevent it from occurring preserving the web application's functionalities.

It is a stored XSS vulnerability. An attacker can send to the victim an email message with malicious Javascript code in the body between `<script>` and `</script>`, and the code will be executed in the victim's browser, in the `secure-webmail.example.com` origin, when the victim loads the message. The attacker can exploit this vulnerability to achieve goals such as reading the session cookie, reading email messages, sending emails on behalf of the victim, The simplest way to prevent the vulnerability is to implement a strict whitelist of allowed tags (e.g., `<b>`, `<i>`, ...) and replace all the other occurrences of special characters with the corresponding HTML entities. Note that using `htmlentities` is not correct, as this would break the requirement of allowing messages with basic formatting.

4. [2 points] Assume that, on the bottom of the above page showing a message, there is a "quick reply" form that allows a logged in user to quickly compose a reply to the email they are reading:

```
<h2>Quick reply!</h2>
<form method="POST" action="send.php">
    <input type="hidden" name="to" value="<?php echo
htmlentities($from); ?>">
    <input type="hidden" name="subject" value="Re: <?php echo
```



```
htmlentities($subject); ?>">
    <textarea name="text"></textarea>
    <button type="submit">Send</button>
</form>
```

Does this implementation of the “quick reply” form introduce any additional vulnerability? Specify which class of vulnerability it introduces, how an attacker can exploit it, what goal can they achieve, and describe the simplest way to prevent it from occurring.

This introduces a CSRF vulnerability. An attacker can exploit it by luring the victim to visit an attacker-controlled page (on any domain); in this page, the attacker has previously placed a piece of Javascript code that auto-submits a form to <https://secure-webmail.example.com/send.php> with attacker-chosen parameters “to”, “subject” and “text”. This way, attackers can send arbitrary emails on behalf of the victim.

The simplest way to prevent it from occurring is to include in the quick reply form a CSRF token, random, unique for each session or for each page visit, and not stored in a cookie, so that the server is able to determine whether the initiator of the POST request was able to read the origin’s data or not.

5.1 [1 point] Consider the XSS vulnerability found above, and consider the web application as a whole (not just the code snippets shown). Describe how you can use the XSS vulnerability to implement a worm-like (malicious) behavior.

I can send to the victim a message with a piece of malicious Javascript code that (a) reads the victim contact list and gathers a set of email addresses; (b) performs some malicious actions (e.g., exfiltrates information); (b) self-replicates by sending an email message containing itself to the gathered email addresses. This email message, when seen by recipients that use the vulnerable webmail, will trigger again the malicious Javascript code, and will self-replicates.

5.2 [2 points] To prevent the spread of a horrible worm, this time the website authors decide to implement a mitigation changing the architecture of the web application. Instead of serving everything from the domain <https://secure-webmail.example.com>, now each email message and attachment are assigned a random token, and are loaded from a different domain, <https://usercontent.secure-webmail.example.com>. No cookies are set for the usercontent.secure-webmail.example.com domain. The message body is then loaded in the page as an iframe:

```
<iframe src=https://usercontent.secure-webmail.example.com?msg=tok>
```

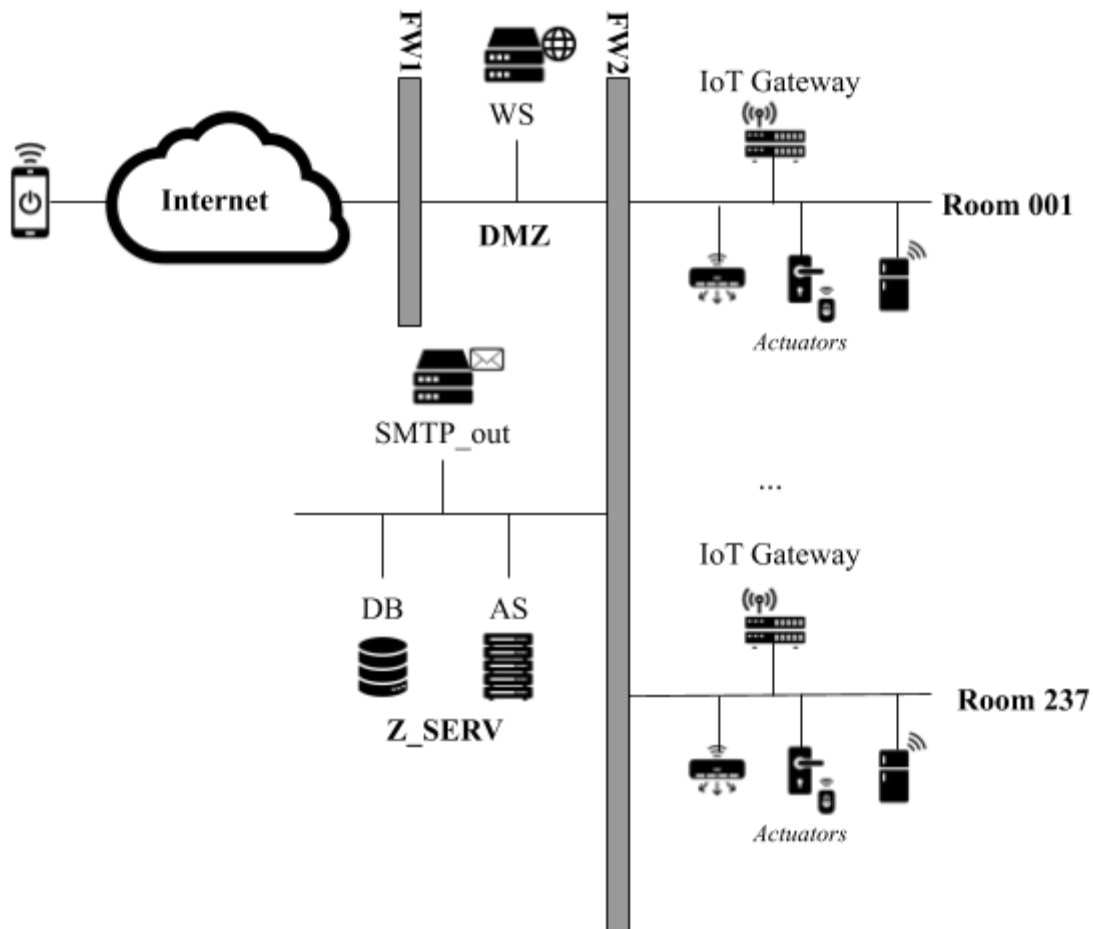
where “tok” is a random, per-message, token (which is securely generated and difficult to guess) assigned to the specific email message to be read.

Can an attacker still exploit the XSS vulnerability with the same effect as before? Why?

Although the XSS vulnerability is still there, an attacker is now able to execute code only in the context of usercontent-secure-webmail.example.com, not in the context of secure-webmail.example.com, and there is no valuable session (or other) information in the former domain. Thus, the attacker is not able anymore to exploit the XSS vulnerability with the same effect.

Exercise 3 (8 points)

The Avengers Hotel has recently adopted a new domotic system that allows to control the appliances inside your hotel rooms (e.g., air conditioner, refrigerator, lights, door locks, gates) from your smartphone over the Internet. For example, you can turn on the air conditioner or open the door when you're approaching the room. The system is composed of a web server, which is connected to actuators (e.g., door lock, light on-off switch) over a local network. You can assume that the actuators are TCP/IP-based devices communicating on port 8080. You send commands to the web server over HTTP (on port 80) via Internet-connected smartphones. The web server forwards the commands on port 9060 towards a centralized Application Server that interprets each command received. The Application Server stores the data on a centralized DB on port 1433 and sends the commands on port 8080 to the respective Room IoT gateway that, in turn, activates or deactivates the actuators. The centralized application server communicates with a local email server to send daily reports to the room owner (on port 25) about the conditions of the room (e.g., temperature). Here is the network layout:



Please, note that:

- Each room has its own subnetwork

1. [2 points] Write the firewall rules, assuming firewalls to be stateful packet filters (i.e. you can consider the response rules implicit) and **considering only one room**.

Firewall	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
FW1 (example)	10.0.0.1 (example)	ANY	zone 1 -> zone 2	192.168.0.2 (example)	443	DENY	(example: the X server in zone 1 cannot contact the Y server)
ALL	ANY	ANY	ANY	ANY	ANY	DENY	Default Deny
FW1	ANY	ANY	WWW->DMZ	WS_IP	80	ALLOW	Allow connection to WS
FW2	WS_IP	ANY	DMZ->Z_SERV	AS_IP	9060	ALLOW	Allow communication between WS and AS
FW2	AS_IP	ANY	Z_SERV->ROOMs	IoTGateway_IPs	8080	ALLOW	Allow communication between AS and IoTGateway
FW2	SMTPout_IP	ANY	Z_SERV->DMZ	ANY	25	ALLOW	Allow to send email
FW1	SMTPout_IP	ANY	DMZ->WWW	ANY	25	ALLOW	Allow to send email

2. [2 points] (1) Identify the most valuable asset and (2) describe a risk scenario against that asset, clarifying the (3) threats and the (4) vulnerabilities that cause it.

The most valuable asset is the actuator of the door lock or gates. In a hypothetical risk scenario an attacker can open the door and break into the house. Specifically, a man in the middle attacker (threat agent) can read the commands and authentication session over the Internet. The vulnerability that caused this attack could be that the network traffic is not encrypted.

3. [1 point] Propose any network-level security mechanism or protocol to ensure that the above risk scenario is properly mitigated.

We propose to use HTTPS

4. After few days, the hotel was hit by a cyber attack: all clients were unable to connect to the web service to access their rooms by using the smartphone.

You are a computer security expert of the S.H.I.E.L.D. (Security Homeland Intervention, Enforcement and Logistics Division). You and your team have been assigned to solve this case and discover what happened. You have obtained the following network logs towards the company web server (with IP 10.133.17.10) through FW1:

Time [seconds]	source	destination	protocol	packet content
0.000	148.79.3.192	10.133.17.10	HTTP	GET / HTTP/1.1
0.002	131.75.7.123	10.133.17.10	HTTP	GET / HTTP/1.1
0.003	10.133.17.10	148.79.3.192	HTTP	200 OK
0.004	110.12.42.3	10.133.17.10	HTTP	GET / HTTP/1.1
0.004	87.15.22.9	10.133.17.10	HTTP	GET / HTTP/1.1
0.005	10.133.17.10	131.75.7.123	HTTP	200 OK
0.005	10.133.17.10	110.12.42.3	HTTP	200 OK
0.006	173.111.23.10	10.133.17.10	HTTP	GET / HTTP/1.1
0.008	10.133.17.10	87.15.22.9	HTTP	200 OK
0.009	141.12.113.33	10.133.17.10	HTTP	GET / HTTP/1.1
0.010	106.139.26.239	10.133.17.10	HTTP	GET / HTTP/1.1
0.011	10.133.17.10	173.111.23.10	HTTP	503 SERVER UNAVAILABLE
0.012	34.101.189.224	10.133.17.10	HTTP	GET / HTTP/1.1
0.012	160.91.121.205	10.133.17.10	HTTP	GET / HTTP/1.1
0.013	10.133.17.10	141.12.113.33	HTTP	503 SERVER UNAVAILABLE
0.014	10.133.17.10	106.139.26.239	HTTP	503 SERVER UNAVAILABLE
0.015	22.55.59.146	10.133.17.10	HTTP	GET / HTTP/1.1
0.016	37.216.29.160	10.133.17.10	HTTP	GET / HTTP/1.1
...				

Tracking the IP addresses contacting the web server (148.79.3.192, 110.12.42.3, 173.111.23.10...), you were able to discover that the vast majority of them came from smart cameras, routers, smart fridges and other IoT devices.

4.1 [2 points] Which attack has been implemented? Give a brief explanation of how it works in general and in this specific case, focusing on what an attacker needs in order to implement them.

DDOS because many IPs are asking to retrieve data, though a botnet probably since the IoT devices... see slides...

The attacker uses a network of (usually compromised) machines to flood the victim with large amounts of unrequested network packets. The attacker needs access to many machines with sizable internet

connections.

Typically, trojans on compromised machines forming a botnet.

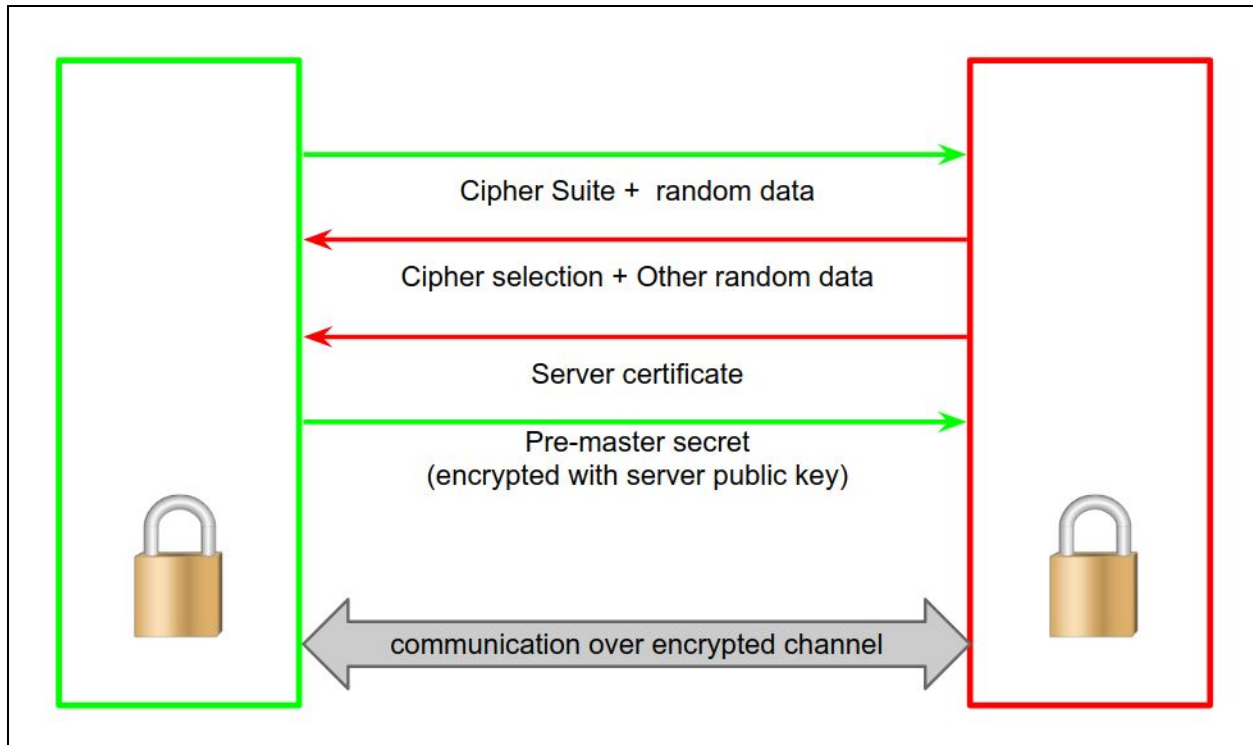
4.2 [1 points] Can you mitigate such attack? Motivate your answer.

Overall, there is no solution to this class of attacks. These attacks can only be mitigated by acting against botnets or by cleaning up infections on compromised machines.

Exercise 4 (5 points)

In a company, each employee has a private office, where only authorized people can enter. The workstation is positioned such that the computer screen faces the wall. Also assume that passwords are only used to access cloud-based services over TLS.

1. [2 points] Consider TLS. Briefly explain how the handshake works and explain why, thanks to the handshake you have just described, it is by design resistant to MITM.



2. We need to design proper policies to minimize the risk that passwords get compromised. Such policies will be enforced whenever a user chooses a new password.

2.1 [1 point] What are the main characteristics of a password on which we can act when writing a policy?

Complexity = length, rich character set

Non guessability = not belonging to dictionaries, not user related

Frequency of change

2.2 [1 point] What is the most likely attack scenario against passwords considering the above description of the conditions of each employee? Why?

Guessing, because the office space is confined and accessible only to authorized personnel. Cracking is the second most-likely attack. Snooping is certainly not an option here.

2.3 [1 point] Given the answer provided in point B., what is the *most important* characteristic that you need to enforce in the password policy (in order to avoid the attack scenario)?

Against guessing, we must enforce that passwords are not related to the user and, in general, not belonging to dictionaries.